ASignX: AI current state, clarifications and suggestions

Repository review | 17 October 2026 | ASignLearn and ASignLive

This report explains what the code and saved model actually do, what the evaluation establishes, what was repaired in the static dataset importer, and how the system can develop into a reliable browser application.

The central finding is that the existing static classifier is a useful, compact baseline. Its main immediate limitation is the quality of the evaluation groups and the coverage of the data. A larger neural network cannot recover movement, body position or signer diversity that the input data never recorded.

READ THIS FIRST

The current classifier is a residual multilayer perceptron (MLP), not an image CNN. Its reproduced test accuracy is 98.14% on the repository's saved split. The split uses imported session IDs that the old extraction code could fabricate, so that figure does not prove accuracy on unseen people or devices. Dynamic recognition has implementation code but no saved dynamic checkpoint or sequence training data was found under ml/data and ml/models.

How to use this report

Sections 1-3 explain the present AI and its data. Sections 4-7 review extraction, cleaning and validation. Sections 8-11 explain results and training. Sections 12-16 compare model choices, body coverage, scalability and bottlenecks. Sections 17-20 describe browser deployment and a practical roadmap. The final sections record verification limits and sources.

Scope

The existing training CSV and production checkpoint were inspected and evaluated, without retraining or rewriting them. Changes concentrate on extraction integrity and an understandable importer interface. Browser and training improvements are explicitly identified as recommendations unless listed as completed repairs.

1. What is the current AI capable of?

There are two machine-learning stages. First, Google's pretrained MediaPipe Hand Landmarker finds hands and estimates 21 points per detected hand. Second, the project's own PyTorch classifier predicts the sign from those coordinates. MediaPipe detects geometry; ASignX supplies the sign vocabulary and learns the sign decision. These are different models with different responsibilities. [W1, W2]

The saved static checkpoint contains 26 labels, A through Z. It accepts a single hand represented by 63 numbers. It can recognize a pose resembling one of those labels; it does not understand arbitrary sign-language words, sentences, grammar or meaning. A classifier with alphabet labels should be described as alphabet/pose recognition until a language-specific vocabulary and independent assessment establish more.

The repository contains 65,798 static samples and 1,665 distinct session strings. Every stored row currently has an ext_ session prefix. Example identifiers include ext_archive3_A_s0000. This naming pattern is consistent with the former extractor's artificial grouping. It does not identify actual recording conditions or signers. The number of unique strings therefore cannot be treated as the number of independent collection sessions.

The current class counts range from 970 for Z to 3,615 for L. Stored handedness is Left for 45,803 rows and Right for 19,995 rows. These are detector labels, not independently verified descriptions of each image. They demonstrate that both orientations appear in the stored data and reinforce the need for matching preprocessing in Python and JavaScript.

Dynamic-sign code supports sequences and two-hand inputs, including an LSTM option and a temporal-convolution-plus-attention option. However, there are no .npz sequence samples under ml/data and no dynamic model checkpoint under ml/models in this checkout. Synthetic training tests establish that the code can execute; they do not establish real dynamic-sign recognition accuracy.

Practical assessment

The static system is a prototype with reproducible internal results and a reasonable small-model design. It is not yet a validated general sign-language recognition or translation system. New-signer testing, unknown-pose testing and actual browser-device measurements are the next evidence needed.

Repository evidence: ml/models/sign_classifier.pth, .metrics.json and .splits.json; ml/data/landmarks.csv; ml/predict.py; ml/sequence_model.py.

2. How the static model works

The current path is: image or camera frame -> MediaPipe hand points -> shared normalization -> learned hand-shape features -> sign scores -> confidence check -> stability check -> displayed/emitted sign.

The saved classifier uses the following architecture:

Stage	Shape / operation	Purpose
Input	21 points x 3 coordinates = 63	Compact hand representation
Feature enrichment	63 + 15 + 10 + 3 = 91	Add joint angles, fingertip distances and palm direction
Encoder stage 1	Linear 91 to 128, LayerNorm, GELU, dropout	Learn combinations of hand measurements
Residual block	Two 128-wide linear layers, normalization and dropout	Refine features while preserving the incoming representation
Encoder stage 2	Linear 128 to 64, LayerNorm, GELU, dropout	Produce a compact sign embedding
Residual block	Two 64-wide linear layers	Refine the embedding
Classifier	Cosine similarity to 26 learned vectors	Produce one score per label
Output decision	Softmax and threshold, then frame stabilization	Suppress weak and brief predictions

An embedding is simply a short list of numbers summarizing the pose. The cosine head compares the direction of this list with a learned vector for each class. A learned scale sharpens or softens the resulting scores. The whole saved model has 63,809 trainable parameters. Its checkpoint is 476,481 bytes, which also includes metadata and split indices; that file size is not the size of a browser deployment bundle.

This classifier is not a convolutional neural network. Its learned layers are dense/fully connected layers, so the correct description is a kinematic residual MLP with a cosine prototype head. The upstream MediaPipe image-processing models use learned visual feature extraction, but that does not make ASignX's separate sign classifier an image CNN. [W2]

The current confidence threshold defaults to 0.8 in SignPredictor. Five consecutive accepted predictions and ten release frames are default stabilization settings. These reduce flicker but do not prove that unfamiliar inputs are rejected. Softmax always distributes probability among the known labels; an unknown hand shape can still receive a high score.

Repository evidence: ml/model.py, ml/preprocessing.py, ml/predict.py and the saved checkpoint's model_config.

3. What landmarks.csv stores and loses

The accepted CSV has exactly 67 columns: label, session_id, handedness, preprocessing, followed by x0,y0,z0 through x20,y20,z20. The coordinate ordering is interleaved by point. It is not all x values followed by all y values and then all z values. Preserving this ordering is essential for training and browser inference.

For a raw hand point p, the static feature is (p - wrist) / maximum wrist-to-point distance. This puts the wrist at zero and makes the farthest point one unit away. The declared preprocessing version is wrist_max_radius_v1. The code checks for 63 finite values, a zero wrist, and a maximum radius near one.

This representation removes global hand location and overall size while retaining the hand's coordinate orientation. It helps recognize the same pose at different positions and distances, but it also discards useful meaning. A hand at the forehead and the same hand at the chin become nearly identical if their local pose and orientation match. A one-frame vector also contains no movement or duration.

The stored depth is the MediaPipe image-landmark depth coordinate, not a sensor measurement of physical distance. Image-normalized x and y use different image dimensions; treating x, y and z as an ordinary Euclidean space is the existing contract, not an exact anatomical reconstruction. Different image aspect ratios and camera viewpoints can therefore shift the feature distribution. Do not switch to world coordinates or pixel-aspect correction in inference alone. That would require a new preprocessing version, regenerated data and retraining. [W1]

The importer now uses the same normalization and numerical validator as training. It also reverses its retry-image padding transform before normalization. Without that correction, added borders could change the relative scale of x, y and z, particularly on non-square images.

The CSV remains a training feature file. Original image hashes, relative source paths, grouping and import decisions are written to an extraction JSON audit beside it. Longer term, keep an immutable raw-data catalog and a separate generated feature store, so changing the landmark model or normalization can trigger a reproducible re-extraction.

Repository evidence: ml/data_io.py, ml/preprocessing.py and ml/extract_dataset.py.

4. Problems found in the extraction program

The program had a useful foundation: recursive discovery, ZIP input, image hashing, shared feature normalization and temporary-file replacement. A complete rebuild of those parts was unnecessary. Several advertised guarantees were stronger than the implementation supported.

Finding	Why it matters	Outcome in this review
Random image buckets called independent sessions	Related images can appear in evaluation and training	Replaced with real/unknown origin grouping
Existing CSV header discarded without validation	Incompatible rows can be relabeled with a new schema	Existing header and rows now validated before import
All split manifests deleted after appending	Destroys evaluation provenance for saved models	Automatic deletion removed
Images copied before detection and cleaning	Rejected inputs appear in the accepted folder	Only accepted candidates are copied
Similarity disabled also disabled exact point deduplication	Reimport can add repeated features	Repeat checks remain active
No record of individual rejection decisions	Hard to investigate missing or mislabeled data	Per-image JSON audit added
Only one detection requested	Two-hand images can silently become one-hand examples	Request two; skip multiple detected hands
Unrestricted ZIP expansion	Unsafe entries, collisions or excessive extraction	Paths, links, encryption and size limits checked
Technical numeric settings in the GUI	Users cannot predict what a threshold changes	Three plain-language cleaning choices

Folder inference was also overclaimed. An algorithm cannot know whether a directory called images2 is a wrapper, a signer or a sign word without a dataset convention. The importer now supports an explicit names file to override guesses. It fails visibly when a names file omits a discovered image or identical files carry different labels.

The legacy prepare_images helper also generated artificial sessions. Its processing chunks now share one origin group per label. The old fix_sessions helper now refuses to rewrite the dataset into fake sessions and explains how to re-import with actual provenance. Existing historical groups were not retroactively rewritten: the source evidence needed to repair them is absent.

Repository evidence: revised ml/extract_dataset.py, ml/extractor_gui.py, ml/prepare_images.py and ml/fix_sessions.py; regression tests in tests/test_extractor_integrity.py.

5. The simpler import workflow

Open ASignX_DataExtractor.bat on Windows, or run python -m ml.extractor_gui in a compatible Python environment. Choose a dataset folder or ZIP, choose the destination CSV and use Preview sign names. The preview runs in a worker thread and shows the inferred labels, image counts and source-group counts. Case variations such as A and a resolve to A.

Common layouts work without physically moving images: dataset/train/A/photo.jpg, dataset/A/photo.jpg and sign_hello/photo.jpg are examples. Copying every input into a second folder is unnecessary for extraction and increases disk usage. The optional accepted-image folder remains available when a reviewer wants a consolidated image collection.

For unusual layouts, provide a Sign names file. It is an ordinary CSV with columns path,label,group. The path is relative to the chosen folder or the archive root; a top-level folder inside a ZIP is part of the path. Every supported image must be listed when this override is used. The label is the intended sign. The group is the original signer or source recording, not a randomly assigned batch number.

Example names file:

```
path,label,group
photos/person_01/image001.jpg,A,person_01
photos/person_02/image002.jpg,A,person_02
photos/person_03/image003.jpg,A,person_03
```

After checking the names, choose a cleaning level and select Extract and add new data. The results show added examples, repeated images, unreadable images, missing hands, multiple hands, unusual poses and the audit-file location. Warnings explain missing independent groups in ordinary language.

An import does not require a user to create three folders. Extraction and evaluation splitting are separate operations. The trainer can split later, but honest splitting requires independent source material. If only one unknown-origin source exists, importing it should still succeed; pretending it contains multiple independent sessions should not.

Use the optional Dataset source name consistently when importing copies from different devices or paths. The default identity is based on the selected path, which cannot establish that two differently named copies came from the same corpus. A stable source name helps maintain this relationship, but reliable provenance still depends on the supplied metadata.

See docs/extractor-guide.md for commands and supported formats. This is a local desktop importer, not a hosted upload service or a mobile executable.

6. How cleaning works and what users should choose

Keep varied examples is the default. The importer skips unreadable images, images with no detected hand, unusable hand geometry, exact file repeats and repeated landmark vectors within a small numerical tolerance. Repeats are checked against both the current import and compatible existing rows of the same label. Old CSV rows are preserved.

Remove very similar examples additionally compares normalized feature distances and removes candidates close to an existing or earlier accepted example. This can reduce a dataset dominated by nearly identical captures. It is a geometric comparison, not an image comparison: two visually different backgrounds can produce the same hand points, while two crops of the same source can produce slightly different points.

Also remove unusual poses applies a per-class statistical filter. It computes the typical coordinate-wise pose, measures candidate distances to it, and uses the median absolute deviation to flag unusually distant examples. This is useful for gross extraction errors, but it is not a correctness test. A valid left hand, uncommon viewpoint, disability-related variation or rare signer style may be far from the dominant cluster. Consequently this filter is optional and every removal is logged.

The bone check only tests broad segment-length plausibility after normalization. It can reject collapsed or extremely distorted predictions. It does not certify anatomical accuracy, the intended sign label, proper finger contact or all occlusion errors. Keep the original images for review.

Better cleaning workflow for a larger corpus

Start with file validity, content hashing and provenance. Cluster related images or augmentations before splitting so families stay together. Quarantine suspicious label conflicts and low-quality detections for human review. Fit any learned outlier threshold on the training partition, then apply the frozen rule without adapting it to held-out data. Record metrics both before and after quality gating, including how many images fail detection.

Do not remove test examples because the classifier finds them difficult. Do not optimize cleaning thresholds on test accuracy. A cleaner dataset should contain fewer errors while preserving genuine variation; a higher score after deleting hard cases does not by itself establish better recognition.

Remaining limits

The current import keeps candidate vectors and the audit in memory. Similarity checks remain quadratic in the worst case. Large-scale extraction should use a persistent source-hash index and chunked or indexed neighbor search, with explicit review of approximate-match errors.

7. Data validation, grouping and honest splitting

read_samples checks the exact header, nonempty labels and session IDs, compatible preprocessing and valid normalized features. It also enforces that one session ID belongs to only one label. The updated importer checks row widths as well, and refuses to append to an incompatible existing file. A temporary file is prepared, flushed and atomically substituted while preserving the previous CSV bytes as the prefix.

The importer holds a cooperative single-writer lock and compares the destination hash before replacement. This protects simultaneous importer runs and detects most outside changes. Other writers, including camera collection, do not yet use this lock. Stop them during extraction; a narrow race remains between the last external-change check and replacement. A crashed process may leave a lock file requiring manual inspection.

The training splitter gathers rows by session ID within each class, shuffles groups using seed 42, and allocates groups to train, validation and test. It requires at least three groups per label. Fractions apply to groups, so row counts need not equal exactly 70/15/15. Its validator checks that each row occurs once, every partition contains every class, sessions do not cross partitions and identical feature bytes do not cross partitions.

These checks cannot establish that two different session IDs represent different people. The historical extractor assigned IDs from row order and chunk size; renamed groups could therefore pass all mechanical checks while still sharing a signer or original recording.

There is another limitation: current sessions must be single-label. A person performing several signs is represented by different IDs, and the splitter handles each label separately. Keeping each label's session intact does not enforce keeping that person out of training across every class.

Recommended evaluation design

Store signer_id, recording_id, source_dataset and original-file/augmentation-family IDs separately. Use a global signer-held-out manifest spanning all labels, with a grouping method such as StratifiedGroupKFold when the data supports it. Keep the final test signers and a separate device/domain test untouched. Grouped splitting is designed to prevent the same group appearing in training and evaluation; it still requires meaningful group metadata. [W3]

Appending data does not mean the existing model learned it. Regenerate the training snapshot and retrain or explicitly register a reviewed prototype. Preserve an immutable final test set so successive releases remain comparable.

8. What the measured results say

The saved model, CSV and split manifest have matching dataset hashes. Running ml.evaluate on that exact test split reproduced the saved metrics, including the confusion matrix and loss. No new production training run was performed.

Measure	Validation	Test
Examples	9,833	9,838
Accuracy	98.4644%	98.1399%
Macro precision	98.3891%	98.1045%
Macro recall	98.5144%	98.2382%
Macro F1	98.4411%	98.1458%
Weighted F1	98.4638%	98.1422%
Cross-entropy loss	0.09025	0.10380

Accuracy is the share of correctly classified examples. Precision asks how often predictions of a particular sign are correct. Recall asks how many actual examples of that sign were found. F1 balances precision and recall. Macro F1 gives every sign equal importance; weighted F1 gives more influence to signs with more examples. The confusion matrix records actual labels by row and predicted labels by column.

The saved training partition contains 46,127 rows. The best checkpoint came from epoch 14 of a 30-epoch run, selected by the lowest validation loss. The default code now allows 50 epochs, so the saved experiment's settings must not be confused with current defaults.

These are closed-set, per-example classification results on already extracted landmarks. They do not include the difficulty of every raw image that failed hand detection. They also do not measure recognition timing, false activations during ordinary conversation, repeated-letter handling or success on a new camera.

The measured result is encouraging evidence that the model can separate these stored classes. Because the source groups were historically synthetic, it is not defensible to describe 98.14% as new-user accuracy. It also does not validate dynamic J or Z movements. A static image classifier can learn a distinctive photographed pose without recognizing the sign's required trajectory.

Do not attach an ordinary independent-sample confidence interval to these rows and treat it as a user-level guarantee. Related images are correlated. Once true signers are known, estimate uncertainty by resampling signers or recordings and reporting the resulting interval.

Repository evidence: saved metrics and manifest; recomputed test results in tmp/review/recomputed-test.json.

9. Where the current classifier makes mistakes

Overall accuracy hides the signs that users will struggle with. The weakest test classes by F1 are shown below. Values are percentages; support is the number of test examples.

Sign	Precision	Recall	F1	Support
T	93.18	92.71	92.95	398
A	88.93	98.31	93.39	237
P	99.33	93.42	96.28	319
X	100.00	93.51	96.65	478
U	97.21	96.47	96.84	397
O	95.44	99.37	97.36	316

The most frequent individual confusions include X predicted as D in 18 cases, T as A in 16, U as R in 13 and I as A in 12. P was confused with Q in 11 examples and T in 10. E was confused with O in 10 examples. These patterns suggest specific collection and review targets rather than a need to increase every network layer.

For A, very high recall combined with lower precision means the system often finds A but also assigns A to other signs. Raising a global confidence threshold might reduce those mistakes while also rejecting correct signs. Threshold selection should therefore use validation data and show the tradeoff between accepted predictions and error rate.

For difficult pairs, review original images with knowledgeable signers: are labels correct, are distinguishing fingers visible, are the landmarks faithful, and is palm orientation preserved? Collect new people performing both members of each pair under varied but plausible viewpoints. Keep these additional recordings separate from the locked final test set.

Recommended live metrics

Report false accepted signs per minute when the user is idle or moving between signs, missed-sign rate, time from a completed sign to emission, repeated-sign accuracy, and lesson pass/fail agreement with human reviewers. Count detector failures and abstentions separately. A system that refuses every input can have few false activations and still be unusable.

Language note

Alphabet and vocabulary definitions must be checked for the actual target language and community. Do not assume labels shared by ASL, FSL or another language have identical required forms or movement.

10. How the current model is trained

Training is supervised multiclass classification. Each normalized hand vector has a known sign label. The network predicts scores, the loss measures disagreement with the label, and backpropagation adjusts the parameters. This is not unsupervised learning, reinforcement learning or training a large language model.

The static saved experiment used batch size 64, initial learning rate 0.002, AdamW optimization, weight decay 0.0001, dropout 0.2 and label smoothing 0.05. A cosine-annealing schedule reduces the learning rate across the run. Inverse-frequency class weights are computed from training rows only. That gives underrepresented classes more influence on the loss.

The training dataset applies augmentation to approximately 75% of retrieved samples. It adds modest 3D rotations and coordinate jitter, then restores the static normalization. Validation and test samples are not augmented. Seeds are set for Python, NumPy and PyTorch, and the split is stored with the dataset hash.

An epoch is one pass through the training set. The current loop evaluates validation performance each epoch and saves the checkpoint whenever validation loss improves. It completes the configured epoch count, then reloads the best checkpoint and evaluates the test set. This is best-checkpoint selection, not early stopping: an early-stopping method would end training after a specified period without improvement.

Assessment of fit

This is a sensible training method for a small landmark classifier. AdamW, mild augmentation, class weighting, dropout and held-out checkpoint selection are reasonable choices. Replacing them merely because another method is newer would not guarantee better recognition. The largest weakness is the independence and representativeness of the evaluation data.

Augmentation also needs language-specific limits. Palm direction and movement can distinguish signs. Broad rotations or mirroring can change meaning, while synthetic coordinate noise is not identical to real occlusion or camera errors. Start with conservative transformations confirmed by sign experts, then compare them with an unaugmented baseline on unseen signers.

For dynamic training, the default config selects a TCN-attention backbone with AdamW and similar regularization. Its so-called speed warping interpolates to an intermediate length and then back over the same normalized duration; this mostly changes smoothing and does not create a substantial real speed change. A future experiment should use genuine time-axis warping, varied clip durations or controlled crop/pad augmentation while preserving labels.

Repository evidence: ml/train.py, ml/train_sequence.py, ml/dataset.py and ml/preprocessing.py.

11. A better training and evaluation method

First fix the experiment, then compare models. Reconstruct provenance from the original datasets where possible. Keep signer, recording and augmentation families intact. Separate a final held-out signer test and a second test representing another device or source dataset. Select the architecture, cleaning rules, augmentation and confidence thresholds only on training and validation material.

Use grouped cross-validation on the development signers to compare at least three candidates: a small plain MLP, the current kinematic residual MLP, and a compact alternative such as a small temporal model for motion or an RBF support-vector baseline for static landmarks. Report macro F1, per-class recall, detection coverage, accepted-prediction error rate and measured inference cost. Repeat promising settings across several seeds. Grouped evaluation guidance supports using meaningful non-overlapping groups; it does not supply the missing signer identities. [W3]

Tune a small, deliberate search space: hidden width, dropout, learning rate, augmentation amount and class weighting. Use early stopping with a validation patience and preserve the best checkpoint. Choose a release criterion that balances macro F1, the weakest important classes and calibration, instead of maximizing overall accuracy alone.

Add real negative examples: hands at rest, ordinary movements, partial hands, incorrect attempts, objects and signs outside the vocabulary. Calibrate the output on validation data and measure false acceptance under domain shift. A temperature parameter can improve probability calibration but does not by itself teach the network to recognize unknown signs. Evaluate an explicit background class or an embedding-distance rejection rule as well.

For dynamic signs, train on complete labeled clips with varied speed and signers, plus non-sign and transition clips. Split before creating overlapping windows. Preserve timestamps, two-hand relationships and missing-data masks. Test on continuous streams with known sign boundaries; classification of isolated clips is only the first milestone.

Use confusion-driven collection. The current T/A, U/R and X/D confusions are priorities for additional valid examples. More unique people and recording conditions usually provide more evidence of generalization than more augmented copies from the same person. Report the effect through a controlled comparison rather than promising an accuracy increase.

Proposed release process

Freeze the data snapshot and group manifest -> run controlled experiments -> select on validation -> assess the final test once -> perform browser parity and device tests -> publish a versioned bundle with its evaluation card. Any new-sign release should recheck old classes and unknown rejection, because adding a class changes the decision problem.

12. Which model types should be considered?

Candidate	Where it fits	Main tradeoff
Small MLP / current residual MLP	Static one-hand alphabet or pose recognition	Cheap inference; cannot see information absent from landmarks
RBF SVM or tree baseline	Small static landmark datasets	Useful comparison; probability and browser packaging need care
GRU or LSTM	Dynamic landmark sequences with modest data	Models time; recurrent execution can limit parallelism
Compact temporal CNN / TCN	Dynamic landmarks and practical client deployment	Efficient window processing; needs timing and boundary handling
TCN plus attention	More varied motion and two-hand relationships	Existing code path; more compute and export complexity
Spatial-temporal graph network	Larger skeleton datasets with meaningful joint structure	Strong architectural match to skeleton connectivity; more engineering
Image CNN or image/landmark hybrid	Signs distinguished by visible contact or details landmarks miss	Higher data, bandwidth and compute requirements

The immediate recommendation for static recognition is to retain the current small residual MLP as a candidate and verify it against a simpler baseline on real held-out signers. There is no measured evidence here that an image CNN will outperform it for the target users. Image-based classification also risks learning backgrounds and collection artifacts.

For dynamic recognition, begin with a compact TCN or GRU/LSTM baseline and compare the repository's TCN-attention model. A TCN is a CNN over time, not an image CNN over camera pixels. The code's TCN blocks use dilated 1D convolutions; attention combines information across a window. The default SignLSTM wrapper can actually select this TCN-attention backbone, so the Python class name is not a reliable description of a trained checkpoint.

A spatial-temporal graph network explicitly models joints and their connections over time. The original ST-GCN research is a relevant architecture reference for skeleton-based action recognition, not evidence that it will automatically improve this sign dataset. A sign-specific comparison must include handshape detail, body anchors, missing joints and device latency. [W5]

Transformer-based sequence models are another option once data volume and sequence complexity justify them. They should not be the first upgrade merely to add capacity. With a small or correlated dataset, a larger model may memorize collection conditions more readily.

For continuous sign language, recognition also needs boundary detection and possibly sequence decoding. Isolated-sign classification alone does not become sentence translation by concatenating labels.

13. Are hands enough, or are arms required?

The required body coverage depends on the signs and the product's claim. Local hand landmarks can be sufficient for a constrained static alphabet whose labels differ in finger configuration and palm orientation. They are insufficient when two signs share a handshape but differ in location, direction, two-hand interaction, contact with the body or nonmanual expression.

Recognition target	Minimum useful signals to evaluate
Static one-hand pose/alphabet	21 hand points, orientation and reliable hand identity
Two-hand static sign	Both hands in a common coordinate frame and hand-presence masks
Movement-based sign	Time-ordered hand points, wrist trajectory, speed and timing
Sign defined near face/chest	Hands plus face/body anchors and body-relative position
Sign involving arm direction	Shoulders, elbows and wrists as well as hand shape
Broad sign-language understanding	Body context and relevant facial/nonmanual signals, plus temporal modeling

Full-arm recognition does not require dense segmentation of every pixel along an arm. Often shoulders, elbows and wrists provide the useful arm pose. A practical next representation is both hands plus upper-body anchors, preserving their relationships and providing confidence/missing-value indicators. Add more signals only when the target sign inventory requires them and a measured comparison shows benefit.

The existing normalize_holistic_landmarks helper returns 81 values: 63 wrist-normalized hand values plus 18 body-relative values. It refers to nose, mouth center, shoulders and elbows. This helper is not connected to the static model's 63-value checkpoint contract, nor accepted by the current sequence loader, which only accepts 63 or 126 features. Defining a helper is not equivalent to collecting body data, training a body-aware model or deploying one.

The sequence normalization is more suitable for movement than the static normalization. It anchors the whole clip to the earliest visible wrist and uses one clip-wide scale. This retains hand trajectory and relative two-hand positions, unlike normalizing each frame independently around its own wrist.

Recommended collection decision

Write a language-specific sign inventory with knowledgeable signers. For each label, record whether finger shape, palm orientation, body location, arm motion, interaction between hands or facial markers distinguish it. This gives an evidence-based choice of camera framing and feature schema. Keep the static importer limited to genuine still-image examples; a photograph cannot validate a required movement.

14. A scalable algorithm architecture

Separate data handling, feature contracts, model training, runtime decisions and user-interface behavior. This lets new signs and devices be introduced without rewriting the whole system.

Recommended offline path

Immutable original datasets -> source catalog and explicit labels/groups -> quality review -> versioned landmark extraction -> frozen grouped splits -> training experiments -> calibrated model -> export and parity checks -> release bundle.

Recommended client path

Camera frames and timestamps -> bounded detection worker -> hand/body tracking -> shared preprocessing -> static pose model and optional temporal model -> confidence/background/boundary decision -> stability and repeat handling -> ASignLearn or ASignLive presentation.

Use an explicit feature contract with version, point ordering, coordinate system, hand slots, missing-data rules, expected sample rate and normalization. Every training snapshot and exported model should declare this contract. The application should refuse a mismatched bundle rather than silently feeding the wrong shape or mirroring convention.

Keep a small static branch for pose recognition. Use a separate temporal branch only when the vocabulary requires motion. Both branches can share detector outputs and feature transforms. A learned temporal/background decision is a better long-term basis for routing than a single wrist-speed threshold. Static signs are often performed while the hand moves into place, and some dynamic distinctions involve finger motion with little wrist movement.

Preserve two canonical hand slots and track identity through time. Include masks for missing/occluded hands; zero coordinates alone can be ambiguous. Add body-relative anchors when required by the inventory. Consider multi-task training for handshape, orientation and movement only if the data has reliable labels for those attributes.

The release bundle should contain the model graph, label mapping, preprocessing version, detector asset version, thresholds, timing configuration, evaluation card and content hashes. A stable label registry should distinguish an internal sign ID from the display name and language. The label A should not be assumed to mean the same thing across unrelated vocabularies.

This architecture scales by growing the dataset catalog and model registry, with explicit compatibility checks. It avoids making a single mutable CSV, one checkpoint filename and hard-coded application defaults the only record of what was trained.

Current versus proposed

The repository already separates many Python modules and shares normalization. It does not yet implement the complete catalog, release registry, global signer grouping, body-aware training or browser inference service described here.

15. Adding new signs without losing old accuracy

The existing cosine head supports adding a class vector from the mean embedding of new examples. SignPredictor.register_new_sign computes this prototype, appends the label and can save an updated checkpoint. This is a useful exploratory capability, but it should not be described as guaranteed accurate training-free expansion.

A new prototype competes with every old class. It can steal predictions from an existing sign or receive poor predictions when the encoder never learned the required distinction. One averaged vector may also represent several viewpoints poorly. The current model is trained with cross-entropy; it is not evidence of a fully validated few-shot metric-learning system merely because its classifier uses cosine similarity.

For production, collect multiple independent signers and recordings of the new sign, including confusing existing signs and negative attempts. Start by testing the frozen encoder and prototype. If performance is inadequate, compare head-only fine-tuning and full fine-tuning while replaying representative old data. A distillation objective or supervised contrastive objective can be evaluated, but these add complexity and require their own experiments.

Every candidate release should rerun the old-class test, new-class held-out signer test, unknown-pose test and calibration check. Compare per-class regressions, not only the average score. Save a new version rather than replacing the only known-good checkpoint. Prototype support data must not double as its evaluation set.

Metadata needs particular care. The registration helper currently carries forward the previous dataset hash and split metadata even though the class list and model have changed. That old evaluation does not evaluate the newly expanded vocabulary. Treat registered prototypes as a new unevaluated model until a fresh evaluation card and matching data snapshot exist.

Browser addition needs a deliberate implementation. A fixed exported ONNX graph and its label/output dimensions do not automatically grow because a Python object changed. Re-export the updated graph, or explicitly export embeddings and implement a separately versioned prototype classifier in the client. The existing JSON tensor dump is not an inference engine and does not implement this workflow by itself.

For a modest alphabet vocabulary, regular retraining with a clean, balanced, versioned dataset is often the simplest reliable approach. Prototype registration can serve as a preview or personalization experiment until its behavior has been assessed across old and new signs.

16. Current and likely bottlenecks

Bottleneck	Effect	Recommended response
Landmark detection on each image/frame	Likely larger cost than the small static classifier	Measure stages; use tracking and avoid redundant frames
In-memory import and per-class distance scans	Large datasets increase time and memory	Persistent hashes, streaming batches and indexed neighbor search
Full CSV replacement when adding rows	Import cost grows with total dataset size	Retain safe snapshots; consider SQLite/Parquet internally and export CSV
Similar viewpoints and unknown provenance	High internal scores, weaker real use	Collect independent people and preserve source families
Occlusion and finger contact errors	Landmark points can omit the discriminating detail	Improve framing; review failures; evaluate a small image/landmark hybrid
Frame-count timing and speed thresholds	Different devices behave differently	Use timestamps, elapsed-time holds and matched sampling rates
Sequence interpolation without explicit masks	Missing hands may look like valid smooth motion	Limit gaps; preserve masks and reject insufficient visibility
Larger attention windows	More memory/compute as window length grows	Compare compact TCN and bounded context first

The static network itself is small. Its 63,809 parameters do not justify assuming that model inference is the dominant runtime expense. MediaPipe, camera capture, conversion, rendering, memory transfer and mobile thermal throttling must be timed separately. No end-to-end device benchmark was performed in this review.

The Python motion gate tracks wrist and fingertips, but its movement measure is displacement per frame, not velocity per second. The TypeScript helper tracks only wrist displacement. Both depend on frame rate and image-space scale. A user moving fingers while holding the wrist still can defeat wrist-only routing.

The default sequence length is 30. Training clips are sampled/resampled through the video path, while live inference accumulates arriving frames. A 30-frame window can represent one second on one device and two seconds on another. Matching the number of frames without matching time is insufficient. Establish a fixed temporal sample rate or a timestamp-aware resampler.

The hand detector suppresses nearby/overlapping detections and stabilizes identities in video mode. This can reduce ghost hands, but legitimate close two-hand interactions need tests as well. Missing or swapped hand identity affects sequence features even when each individual point estimate appears plausible.

Avoid premature optimization of the classifier. First instrument detection coverage, per-stage latency, queue depth, dropped frames, memory and temperature-related slowdown on the intended devices.

17. Can recognition run entirely in the website?

Yes, the proposed split is practical: train offline, publish model assets, and run camera detection plus sign inference locally in the browser. MediaPipe provides a JavaScript Hand Landmarker API. ONNX Runtime Web provides browser inference with a WebAssembly path and optional GPU execution, subject to operator/backend support. These are deployment building blocks, not proof that the repository's current graph already runs correctly on every device. [W1, W4]

Use HTTPS for production camera access. Ask for camera access when the user starts the feature and provide stop/retry controls. Keep the mirrored preview separate from the inference coordinate convention. Run detection and inference in a worker or otherwise keep blocking work off the main interface thread. Google's web guide specifically notes that synchronous detection calls can block the interface and suggests workers. [W1]

Start with the small static model on WebAssembly. Measure before choosing WebGPU: a GPU path can add overhead for a very small model and support varies. For a larger temporal model, compare compatible backends on the target browser matrix. Select a tested fallback and surface an actionable error if initialization fails. [W4]

The client should load a versioned model bundle, verify its metadata, transform MediaPipe image landmarks exactly as training did, run inference, and pass logits to a common decision layer. The exported static model already contains kinematic enrichment. Feed it 63 normalized floats; do not expand to 91 in JavaScript and then feed an incompatible graph that expands them again.

Use one model session per required model and reuse it. Process the newest frame when the worker is free; do not build an unbounded queue of stale frames. Reuse typed arrays where practical, release resources on navigation, and cache only compatible versioned assets. Camera images can remain local if the application sends no frames or landmarks to a server; review telemetry and third-party code before making a privacy claim.

A separate browser dataset extractor is feasible as a future tool: select local image files/ZIPs, run the same static detector in image mode, preserve names/groups, clean samples, and download a new CSV or append through explicit file access where supported. Browser file access and mobile folder selection differ, so ZIP input plus downloadable output is a useful fallback. This review did not implement that browser extractor.

18. Browser gaps that must be resolved

The existing web_client/asignx_client_sdk.ts is a collection of feature/coaching/stability helpers. It does not load an ONNX model, initialize MediaPipe, manage a camera or implement the complete ASignLearn/ASignLive website. The Python exporter also needs a reliable web-success contract.

First, there is a concrete preprocessing mismatch. normalizeHandLandmarks in TypeScript mirrors x for Left hands. Python normalize_landmarks does not. With the current mixed-handedness training data, deploying that JavaScript behavior changes the input distribution for left hands. Remove the extra client mirroring for this checkpoint, or introduce a new canonicalized preprocessing version and regenerate/retrain everything consistently. Mirroring only the visual preview is a separate UI choice.

Second, export_static_checkpoint and export_sequence_checkpoint catch any ONNX export exception and silently return a TorchScript fallback. TorchScript is not an ONNX Runtime Web graph. The current tests verify the existence of metadata and JSON weights; they do not prove that ONNX export succeeded or that a browser can execute the graph. A JSON state_dict dump similarly has no general browser inference implementation here.

Third, metadata is incomplete for a strict client contract. Static export should declare preprocessing, coordinate conventions, detector asset identity, actual architecture, output labels and calibrated thresholds. The TypeScript interface expects hidden_dim, whereas static export provides hidden_sizes. Build a single schema rather than relying on assumptions between loosely related files.

Fourth, the Learn helper increments a hold counter without automatically resetting it on a target-label change. It also reports zero finger error when no reference centroid exists, which can make unavailable diagnostics look like correct alignment. Reset state when the target or tracking identity changes, and represent unavailable coaching explicitly.

Fifth, the Live helper can return an old stableLabel while the current prediction is null or moving; emitting a new label and displaying current stability are different concerns. Its repeat-release logic depends on missing-hand calls, and wrist-only motion differs from Python. Define and test one state machine, including repeated letters separated by neutral poses.

These browser issues were identified for the deployment plan; the SDK and export code were not rewritten in this extraction-focused change. Before deployment, require successful ONNX generation, runtime numerical parity and real browser tests. Do not substitute a fallback file and call deployment complete.

19. ASignLearn and ASignLive need different decisions

ASignLearn knows the target sign. It can compare the attempt with that target, show an example, track a steady hold and provide a clear retry. Use elapsed milliseconds for the hold so a low-frame-rate phone does not require a longer physical hold than a desktop. Reset when the target changes, the hand disappears or tracking switches to another hand.

Coaching should explain observable differences rather than present classifier confidence as anatomical correctness. A class median can average incompatible viewpoints into an unrealistic reference. Prefer curated reference poses or several valid templates separated by orientation, and use joint-angle or relative-distance differences when suitable. Say that feedback is unavailable when the reference or tracking quality is inadequate. Validate hints with knowledgeable signers and representative learners.

Lesson success should combine target probability, sufficient visibility, appropriate orientation/location where required and a stable duration. If the lesson teaches a movement-based sign, a static pose match cannot be the success criterion. Offer user-understandable messages such as Bring your hand fully into view or Hold this pose steady when the corresponding condition is actually measured.

ASignLive does not know the next sign. It must distinguish signs from rest, transitions and unrelated gestures. Use a background/unknown decision, temporal evidence, start/end boundaries and hysteresis so confidence does not switch rapidly near a threshold. Keep separate state per tracked hand for independent one-hand recognition, and a joint two-hand state for signs whose meaning depends on interaction.

Repeated signs require an explicit policy. A held A should emit once; A, neutral, A should be able to emit twice. Changing to another sign should not require a hand to leave the frame. Clear a displayed stable sign when evidence becomes insufficient, while retaining the separate event history as needed.

For dynamic signs, retain timestamps and a bounded window with matched sample rate. The current symmetric TCN convolutions and full-window attention are not an incremental causal streaming model. They can be used on a completed/sliding window, but latency and gesture boundaries must be measured. A causal TCN or another incremental model is an alternative if low delay becomes a requirement.

Acceptance tests should cover slow and fast users, left and right hands, repeated signs, interruptions, occlusion, camera changes, idle movement and switching lessons. A high offline alphabet accuracy does not replace these product-level tests.

20. Deployment gates and implementation roadmap

Phase 1: use the repaired static importer. Preview labels, preserve actual source groups, keep a source audit and review difficult/ambiguous images. Preserve old models and manifests as evidence. Do not automatically rewrite the 65,798-row dataset into supposedly better sessions without the original provenance.

Phase 2: establish a trustworthy benchmark. Collect or recover signer/recording identities, create a global group manifest, freeze an independent test set and compare static baselines. Add unknown/incorrect attempts and detector-coverage reporting. This phase should precede claims of improved user accuracy.

Phase 3: build the minimal browser static feature. Fix Python/JavaScript preprocessing parity, make ONNX export fail clearly, implement camera/worker/model loading, and provide a WASM-first tested path. Pin compatible runtime and detector assets. The existing Windows binary is not a route to mobile browser deployment.

Required numerical checks

Create a golden set of raw 21-point hands and image examples, including both handedness labels and non-square frames. Compare Python and browser normalized vectors; compare exported-graph logits, predicted labels and calibrated confidence on the same inputs. Choose and record numerical tolerances. Test that the graph accepts 63 floats and that label order matches output order. Run the comparison again after quantization or backend changes.

Required runtime checks

Measure camera-to-decision latency and p50/p95 stage times, sustained frame rate, memory and thermal behavior on representative phones and laptops. Verify camera permission denial, loss/recovery, navigation cleanup, offline asset failure and version mismatch. Browser GPU support alone does not establish operator compatibility or speed. [W4]

Phase 4: add validated body-aware/dynamic recognition only where the sign inventory requires it. Collect sequences, preserve timing and masks, train a compact temporal baseline, then compare the existing attention option. Evaluate continuous streams and repeated-sign behavior. Do not claim dynamic readiness from synthetic unit tests.

Phase 5: introduce controlled new-sign releases, dataset/model registries and collection prioritization based on observed errors. Keep a known-good release available and monitor regressions across existing classes and devices.

Suggested release gates are product decisions, not measured guarantees: no unexplained preprocessing mismatch, no missing export artifact, acceptable per-sign error/coverage, acceptable false activations and a latency budget demonstrated on supported devices. Set numerical targets with stakeholders and sign-language reviewers before the final test.

21. What was verified and what remains uncertain

Completed code changes include origin-preserving extraction, explicit names-file overrides, destination validation, accepted-only image copies, ZIP validation, animated-image detection, repeat checks, audit reports and plain-language cleaning presets. The source grouping behavior was also corrected in the legacy image helper, and the destructive fake-session repair helper now refuses to run.

Verification includes regression tests for incompatible CSV protection, existing-byte preservation, wrong preprocessing, changed destinations, repeat imports, source-group stability, malicious archive paths, explicit mappings, conflicting labels, lock behavior, animated-image handling and padding coordinate restoration. All 54 repository tests passed, with 10 tracing/deprecation warnings from the existing export tests. The GUI was constructed successfully through Tkinter, and the real MediaPipe detector processed a blank non-square image without detecting a hand.

The existing model was evaluated on its saved test split and its reported scores were reproduced. The dataset hash matched the checkpoint and split manifest. No production retraining was performed. The original training dataset, checkpoint and saved metrics were not changed by this review.

Boundaries of this evidence

Most extraction regression tests use controlled fake detector outputs to isolate data integrity. The blank-image smoke test checks detector initialization and a no-hand case, not extraction quality on a real corpus. No representative raw dataset was provided for a before/after detector-coverage experiment. No camera/device latency benchmark or end-to-end mobile browser trial was performed.

The importer can validate file structure and numerical features; it cannot confirm that a label is linguistically correct, that a still image depicts a truly static sign, or that a supplied group represents an independent person. Nearby two-hand detections can also be affected by the detector's duplicate suppression. Human review and appropriate collection remain necessary.

The Windows packaging uses the installed Python 3.11 environment, whose observed packages include MediaPipe 0.10.21, OpenCV 5.0.0 and NumPy 2.4.6. That is not the same as proving a clean installation from the project's Python 3.14-oriented requirements.txt. A Windows package must be tested on a clean target machine before broader distribution. Copy its entire folder, not just the executable.

The program has no cross-platform binary or hosted upload service. It runs locally through compatible Python or the supplied Windows build. Client-side web extraction and recognition are future implementations described in this report. Similarity pruning and audit storage still have scaling limits, and the cooperative lock does not fully coordinate unrelated CSV writers.

22. Sources and reproducibility

Repository evidence was inspected directly on 17 October 2026. References below distinguish the project implementation from external documentation and architecture research. Recommendations in this report are engineering judgments for this codebase, not measured promises of higher accuracy.

R1. ml/models/sign_classifier.pth: actual architecture, labels, training configuration, parameter values, dataset hash and split indices.

R2. ml/models/sign_classifier.metrics.json and sign_classifier.splits.json: recorded epoch selection, per-class metrics, confusion matrices and fixed test rows.

R3. ml/data/landmarks.csv; ml/data_io.py; ml/dataset.py; ml/preprocessing.py: sample counts, normalization and validation/split contracts.

R4. ml/model.py; ml/train.py; ml/evaluate.py; ml/predict.py: static architecture, optimization, metrics and runtime decision logic.

R5. ml/sequence_model.py; ml/sequence_dataset.py; ml/train_sequence.py; ml/video.py; ml/predict_sequence.py: implemented dynamic sequence contracts and unvalidated real-data capabilities.

R6. ml/extract_dataset.py; ml/extractor_gui.py; ml/prepare_images.py; ml/fix_sessions.py; docs/extractor-guide.md: extraction review and completed repairs.

R7. ml/export_onnx.py; web_client/asignx_client_sdk.ts: existing export behavior, preprocessing mismatch and browser implementation gaps.

W1. Google, Hand landmarks detection guide for Web. JavaScript task setup, synchronous processing behavior and image/world landmark outputs. https://developers.google.com/edge/mediapipe/solutions/vision/hand_landmarker/web_js

W2. Zhang et al., MediaPipe Hands: On-device Real-time Hand Tracking (2020). Upstream hand-detection/landmark pipeline. https://arxiv.org/abs/2006.10214

W3. scikit-learn, Cross-validation: evaluating estimator performance, grouped iteration. Group separation and stratified grouped validation. https://scikit-learn.org/stable/modules/cross_validation.html

W4. ONNX Runtime, Get started with ONNX Runtime Web and Using WebGPU. Browser runtime choices, supported deployment paths and tradeoffs. https://onnxruntime.ai/docs/get-started/with-javascript/web.html and https://onnxruntime.ai/docs/tutorials/web/ep-webgpu.html

W5. Yan, Xiong and Lin, Spatial Temporal Graph Convolutional Networks for Skeleton-Based Action Recognition (2018). Candidate architecture reference, not a sign-recognition result for this repository. https://arxiv.org/abs/1801.07455

Reproduction command: py -3.11 -m ml.evaluate --output tmp/review/recomputed-test.json. The test run uses the existing checkpoint and fixed test manifest; evaluation does not retrain the model. Start the revised importer with ASignX_DataExtractor.bat or python -m ml.extractor_gui.